Deep Learning: Advanced Architectures

M23DES313 Deep Learning · Unit 1 · Exam Prep Notes (easy language)
Based on the Unit 1 slides (Dr. Ambili P S). These slides cover neural network basics, CNN, object detection (YOLO, SSD), RNN, LSTM and GRU. Transformers/Attention, GANs and VAEs are in the syllabus but not in these slides.

> **Unit 1 at a glance:** Neuron & Perceptron → Multilayer network → CNN (convolution, filters, activation, padding, pooling, dense layer) → Object detection (YOLO, SSD) → RNN → LSTM → GRU.

1. Introduction to Neural Networks

- An **artificial neuron** copies a biological neuron: inputs (like dendrites) come in, are combined, and one output goes out.
- Each input **x** has a **weight w**. The neuron adds up weighted inputs plus a **bias b**, then passes the sum through an **activation function f**.

> **Formula:** $y = f(\Sigma w_i x_i + b)$ (sum over i = 1 to n inputs)

Perceptron (single neuron)

- Basic architecture: **input nodes** ($x_1 ... x_n$) → weights ($w_1 ... w_n$) → **summation (Σ)** + **step activation** → output y.
- Can be drawn **without bias** or **with a bias neuron (+1)** having weight b.

Multilayer neural network

- Has an **input layer**, one or more **hidden layers** and an **output layer**. Every neuron connects to all neurons of the next layer.
- Can be built with or without **bias neurons** in each layer.

2. Convolutional Neural Network (CNN)

A CNN works on images. **Flow:** Input image → [Convolution + ReLU → Pooling] repeated → Flatten → Fully connected layer → Softmax → Output.

Part	Job
Feature extraction (Conv + ReLU + Pooling layers)	Finds edges, corners, shapes, and then objects in the image
Classification (Flatten + Fully connected + Softmax)	Decides the class label of the image

2.1 Convolution operation

- A small matrix called a **filter / kernel / feature detector** slides over the input image. At each position, multiply matching numbers and add them up. The results form a **feature map**.
- Many filters can be used on one image (each gives its own feature map).

> **Output size:** input (n x n) * filter (f x f) = **(n − f + 1) x (n − f + 1)**.
> Example: 6x6 image with 3x3 filter gives a 4x4 feature map.

Worked example from slides (filter = [1 0 −1 / 1 0 −1 / 1 0 −1], image 6x6 starting 1 6 9 ... / 2 5 1 ... / 3 7 4 ...):
Step 1 (top-left): 1×1 + 2×1 + 3×1 + 6×0 + 5×0 + 7×0 + 9×(−1) + 1×(−1) + 4×(−1) = **−8**.
Step 2 (move one step right): 6×1 + 5×1 + 7×1 + 9×0 + 1×0 + 4×0 + 10×(−1) + 8×(−1) + 9×(−1) = **−9**. Continue until the whole 4x4 map is filled.

2.2 Filters and their function

- **Feature detection / extraction:** filters are pattern detectors (horizontal edges, vertical edges, complex structures).
- **Learning representations:** different filters learn different patterns during training. The network builds features **from simple (edges) to complex (objects)**.
- Example filters: vertical line filter, diagonal line filter, "loopy pattern" filter (for digit 9).
- **Location invariant:** because the same filter moves across the whole image, it can detect a feature (e.g. an eye) **anywhere** in the image.

2.3 Activation function

- The hidden layer computes on the input features and passes the result on. The **activation function decides whether a neuron is activated** (after weighted sum + bias).
- The network updates weights and biases from the output error. This is **back-propagation**. Activation functions make it possible because gradients are passed along with the error.
- Without an activation function, a network is just **linear regression**. Purpose: introduce **non-linearity**.

Function	Equation	Range	Key points
Linear	$f(x) = x$	$(-\infty, \infty)$	No non-linearity
Sigmoid	$1 / (1 + e^{-x})$	(0, 1)	Used in output layer for **binary classification** (>0.5 or <0.5)
Tanh	$(e^{x} - e^{-x}) / (e^{x} + e^{-x})$	(−1, 1)	Zero-centered; used in hidden layers
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	Most used in hidden layers; **overcomes vanishing gradient**
Leaky ReLU	0.01x if x<0, else x	$(-\infty, \infty)$	Small slope for negative values
ELU / SELU	$\alpha(e^{x}-1)$ if x≤0, else x	$(-\alpha, \infty)$	Variants of ReLU
Softplus	$\ln(1 + e^{x})$	$(0, \infty)$	Smooth version of ReLU
Softmax	$e^{x_i} / \Sigma\, e^{x_j}$	(0, 1), sums to 1	A sigmoid-type function for **multi-class classification**; used in the **output layer** of image classifiers

> **Vanishing gradient problem:** for sigmoid and tanh, at very large or very small input the curve is flat, so the derivative (gradient) becomes tiny and **learning becomes very slow**. ReLU avoids this for positive values.
>
> Note: the slides' range column shows (0,1) for Softplus and (−1,1) for Leaky ReLU, which look like typing slips; the ranges above are the standard ones.

2.4 Padding

- Adding extra border values around the input so edge information is not lost. It **reduces loss of information at the borders** and can improve performance.
- **Zero padding** (border of 0s) is the most common. **Replication padding** copies the edge values. **No padding** = image shrinks after convolution.

2.5 Pooling

- Uses a filter/kernel (e.g. 2x2, stride 2) to **shrink the feature map**.
- **Max pooling:** takes the **highest** value in each region (picks most salient features). **Average pooling:** takes the **average** of each region (keeps more information).
- **Output size:** (N − F) / stride + 1 (e.g. 4x4 map, 2x2 kernel, stride 2 gives 2x2).

Why use pooling?

- Reduces dimensions (spatial size) of feature maps.
- Reduces number of parameters and computation.
- Summarizes features of a region, so the model is **robust to changes in position** of features.
- Selects the most important features.

2.6 Fully connected (dense) layer

- Conv + pooling extract **spatial features** (edges, textures, shapes, high-level patterns like faces) but they are still in a **2D grid**. So the maps are **flattened** into a 1D vector and fed to dense layers.
- Every neuron connects to **every neuron** in the next layer, so signals mix and the decision uses the **whole image**.
- It **classifies the output** and **associates features with a particular label**. Each neuron: weighted sum + bias → activation. Last layer uses **softmax** to give class probabilities.

Examples in slides: Deep CNN classifying a car vs an elephant (softmax gives probabilities such as sports car, African elephant); and **brain tumour classification** into meningioma (tumour of the meninges, the tissue layers covering brain and spinal cord), glioma (at the base of the brain) and pituitary (tumour in the pituitary gland, which controls body growth).

3. Object Detection

- **Object detection** = a computer vision task that **identifies objects in an image** and **locates them using bounding boxes**.
- It combines **image classification + localization** (and in some methods also instance segmentation).
- **Classification** says "Cat" for the whole image; **detection** says "Cat, Dog, Dog" and where each one is.

Applications

Self-driving cars (vehicles, lanes, pedestrians) · Security (people, weapons, faces) · Healthcare (tumour detection) · Retail (product detection) · Agriculture (plant disease) · Robotics (navigation, obstacles).

Two types of detectors

Two-stage detectors	One-stage detectors
Slow but highly accurate. Stage 1: region proposal (create object candidates). Stage 2: classify and refine each candidate. Examples: R-CNN (2014), Fast R-CNN and Faster R-CNN (2015).	**Fast, real-time.** Skip region proposal. Predict bounding boxes + class probabilities **directly from the whole image in a single pass**. Examples: YOLO, SSD.

4. YOLO (You Only Look Once)

- A **real-time, one-stage** detector that does **classification + localization in a single forward pass**.
- Used in surveillance, traffic detection, robotics, retail automation, agriculture/drones, medical imaging.
- **Versions:** YOLOv1 (2016), v2/YOLO9000 (2017), v3 (2018), v4 and v5 (2020), v6 and v7 (2022), v8 (2023), v9, v10, v11 (2024).
- **Architecture** is similar to GoogLeNet: **24 convolutional layers, 4 max-pooling layers and 2 fully connected layers**.

YOLO works on 4 ideas

1	2	3	4
Residual blocks	Bounding box regression	IoU	Non-Max Suppression

1. Residual blocks (grid)

- The image is divided into an **N x N grid** of equal cells (N = 4 in the slide example). Each cell **localizes and predicts the class** of the object it covers, with a confidence value.
- **Why residual blocks?** Faster training, better feature extraction, higher accuracy on small/complex objects.

2. Bounding box regression

- Finds the rectangles that highlight objects, using one regression module. Each box is described by the vector:

> **Y = [pc, bx, by, bh, bw, c1, c2]**
> pc = object probability (1 if object present); bx, by = x, y coordinates of box centre; bh, bw = height and width; c1, c2 = one-hot class values (e.g. c1 = Dog, c2 = Person).
> Slide example: Y = [1, 50, 70, 70, 60, 1, 0] means an object is present, at that position/size, and it is a dog.

- YOLO **learns to fit the box closest to the object's true size and location** (regression).

3. Intersection over Union (IoU)

> **IoU = Area of Overlap / Area of Union** (between ground-truth box and predicted box).
> Value from 0 to 1; higher is better.

- Compares the **ground truth box** with the **predicted box** to judge detection quality.
- Helps to: compute **loss** during training; decide **how good** a predicted box is; **filter out** incorrect predictions.

4. Non-Maximum Suppression (NMS)

- Many boxes may detect the same object. NMS **removes redundant boxes based on confidence scores** and keeps only the best one.
- Benefits: prevents overlapping boxes; each object gets **only ONE final box**; improves detection clarity.

5. SSD (Single Shot MultiBox Detector)

- Architecture: a **base network (VGG-16)** for feature extraction + extra convolutional layers for **multi-scale detection**.
- **VGG-16:** CNN from the Visual Geometry Group, Oxford; **16 layers = 13 convolutional + 3 fully connected**.

Core idea

Word	Meaning
Single shot	Everything happens in one pass of the network
MultiBox	Many boxes (anchors / default boxes) predicted at every location
Detector	Each box gives **class scores** (car, person, dog...) and **box offsets** (x, y, w, h)

Steps

1. Input image is resized and passed into a base CNN (VGG-16 or ResNet).
2. SSD takes **feature maps from multiple layers** of the backbone.
3. **Default boxes (anchors)** are placed at each cell of each feature map. (YOLO has one N x N grid; SSD has grids at **multiple scales** and combines all predictions.)
4. For every default box, SSD predicts a **class score vector (including background)** and **offsets** to refine the default box into the final box. Multiple boxes cover different object sizes and aspect ratios.
5. **NMS**: SSD makes thousands of predictions. Sort boxes by confidence → keep the highest-score box → remove boxes with high IoU overlap.

YOLO	SSD
One N x N grid on the image	Multiple feature maps at different scales
Backbone similar to GoogLeNet (24 conv layers)	Backbone VGG-16 (or ResNet)
Predicts [pc, bx, by, bh, bw, classes] per box	Predicts class scores + offsets for default boxes
Both are one-stage, fast, and use IoU and NMS	

6. Recurrent Neural Network (RNN)

- A deep learning model that processes **sequential data** (words, sentences, time series, speech) where **order matters**, and converts it into sequential output.
- It has a **memory** that stores information about previous steps. Example use: translating text from one language to another.

- Today RNNs are largely being replaced by **transformer-based AI and LLMs**, which are more efficient for sequences.

Key components

1. Recurrent neurons

- Basic unit with a **hidden state** that holds information about previous inputs. The hidden state is **fed back**, so information cycles through a loop. A decision uses the **current input + what was learned from earlier inputs**.
- Symbols: **x** = input at time t; **h** = hidden state (memory); **L** = output at time t; **U** = input weight matrix; **V** = recurrent weight matrix.

2. RNN unfolding

- Expanding the recurrent structure **over time steps**; each step becomes a separate layer in a series.
- This allows **Backpropagation Through Time (BPTT)**: errors are propagated across time steps to adjust weights, so the RNN learns dependencies in sequences.

> **At each time step t:**
> (a) Input x_t enters. (b) Hidden state updated: $\mathbf{h_t = f(U\, x_t + V\, h_{t-1})}$ (h_{t-1} = memory from previous step; U, V = weights). (c) Output: $\mathbf{L_t = W\, h_t}$.

Example: sentence "He is a fantastic actor" is fed one word at a time: X0 = He, X1 = is, X2 = a, X3 = fantastic, X4 = actor.

tanh in RNNs: commonly used because its output is **zero-centred and bounded (−1 to +1)**, which stabilizes hidden states, improves gradient flow during BPTT, and represents both positive and negative dependencies. Good for language modelling.

Types of RNN

Type	Input	Output	Application
One-to-One	Single	Single	Image classification
One-to-Many	Single	Sequence	Image captioning
Many-to-One	Sequence	Single	Sentiment analysis
Many-to-Many (same length)	Sequence	Sequence	POS tagging
Many-to-Many (different length)	Sequence	Sequence	Machine translation

7. LSTM (Long Short-Term Memory)

- **Problem:** normal RNNs struggle to remember information over long sequences (**vanishing gradient**). Example: "France is the place I grew up. I speak fluent ____" needs the word "France" from far back.
- **LSTM** is a type of RNN with **memory cells and gates** that selectively **store, forget and output** information, so it can learn **long-term dependencies**.
- Difference from RNN: an RNN has a single **tanh** layer; an LSTM has a **gated cell** with **three sigmoid gates and one tanh layer**.

Key components

Component	Role
Cell state (C_t)	**Long-term memory.** Information flows with minimal change; gates add or remove information.
Hidden state (h_t)	**Short-term memory.** Output of the LSTM cell at time t.

Gate	Job	Activation
Forget gate	Removes unnecessary information from the cell state	Sigmoid
Input gate	Adds new information	Sigmoid + Tanh
Output gate	Produces the output (hidden state)	Sigmoid + Tanh

> **Forget gate formula:** $f_t = \sigma(\ W_f\ [h_{t-1}, x_t] + b_f\ )$ Output range 0 to 1: **0 = forget completely, 1 = keep completely**.

LSTM cell diagram (from slides): inputs C_{t-1}, h_{t-1}, x_t → forget gate (sigmoid) multiplies the old cell state → input gate (sigmoid + tanh) adds new information (point-wise addition) → output gate (sigmoid) multiplies with tanh of the cell state → gives h_t and new C_t.

Applications

Language modelling · speech recognition · time-series forecasting · anomaly detection · recommender systems · video analysis.

8. GRU (Gated Recurrent Unit)

- A **simplified LSTM** that handles long-term dependencies with **fewer parameters and faster training**.
- **Differences from LSTM:** no separate cell state (C_t); no output gate; memory is stored directly in the **hidden state**.

LSTM	GRU
3 gates: **Forget, Input, Output**	2 gates: **Update, Reset**
Has cell state and hidden state	Only hidden state
Finer control over memory; good for complex sequences	Fewer parameters, faster training
More reliable for **very long** sequences	Simpler

9. Last-Minute Revision

- **Neuron:** y = f(Σwx + b). Activation adds **non-linearity**.
- **CNN flow:** Conv + ReLU → Pooling → Flatten → Dense → Softmax. Conv output = n − f + 1. Pooling output = (N − F)/stride + 1.
- **Filters** detect features and are **location invariant**. **Padding** keeps border information. **Max pooling** = highest value; **average pooling** = mean.
- **Sigmoid** = binary output; **Softmax** = multi-class output; **ReLU** fixes vanishing gradient; **tanh** range −1 to 1.

- **Object detection** = classification + localization. Two-stage (R-CNN family) = accurate, slow; one-stage (YOLO, SSD) = fast.
- **YOLO:** 24 conv + 4 max-pool + 2 FC; grid, bounding box Y = [pc, bx, by, bh, bw, c1, c2], IoU = overlap/union, NMS removes duplicate boxes.
- **SSD:** VGG-16 backbone, multi-scale feature maps, default boxes, NMS.
- **RNN:** $h_t = f(Ux_t + Vh_{t-1})$, trained with BPTT, uses tanh. 5 types (one-to-one, one-to-many, many-to-one, many-to-many x2).
- **LSTM:** cell state + forget/input/output gates; solves vanishing gradient. **GRU:** update + reset gates, no cell state.